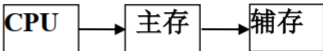
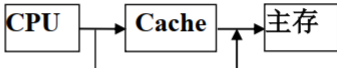




第 5 章 Pentium 微处理器保护模式存储管理

- **5.1 虚拟存储器及其工作原理**
- **虚拟存储器：**由主存储器、辅助存储器、辅助硬件和操作系统管理软件组成的一种存储体系。
- **虚拟存储系统目标：**增加主存储器的存储容量。它的速度接近于主存，单位造价接近于辅存，因此性能价格比很高。

表 5.1.1 虚拟存储器和 Cache 存储器的比较

存储体系	虚拟存储器	Cache存储器
存储层次	主存-辅存	Cache-主存
主要功能	主存速度，辅存容量	CPU速度，主存容量
信息传送单位	信息块（比如 段 、页），有多种划分，长度较大。	信息块（比如 块 、行），长度较小，并且固定。
结构差别	 主存不命中，进行 辅存调度 ，CPU程序换道。	 在CPU与主存之间具有 直接访问 通路。
操作过程	由部分 硬件 和操作系统存储管理 软件实现 ， 对应程序员透明 ，对存储管理软件程序员不透明。	全部用 硬件实现 ，对各类程序员透明。



5.1.1 地址空间及地址

在虚拟存储器中有3种地址空间及对应的3种地址：

- **虚拟地址空间：** 又称为虚存地址空间，是应用程序员用来编写程序的地
址空间，与此相对应的地址称为虚地址或逻辑地址。
- **主存地址空间：** 又称为实存地址空间，是存储、运行程序的空间，其相
应的地址称为主存物理地址或实地址。
- **辅存地址空间：** 也就是磁盘存储器的地址空间，是用来存放程序的空间
，相应的地址称为辅存地址或磁盘地址。



5.1.2 虚拟存储器工作原理

存储管理方式:

- 由于采用的存储映象算法不同，形成不同的存储管理方式，其中主要有**3种：段式、页式、段页式**
- **Pentium**支持分段存储管理、分页存储管理和段页式存储管理。

Pentium微处理机的存储管理部件:

- 由分段部件和分页部件组成。
- **分段部件功能：**将逻辑地址转换成一个连续的不分段的地址空间，这个地址空间的地址叫做**线性地址**。
- **分页部件功能：**将线性地址转换成物理地址。

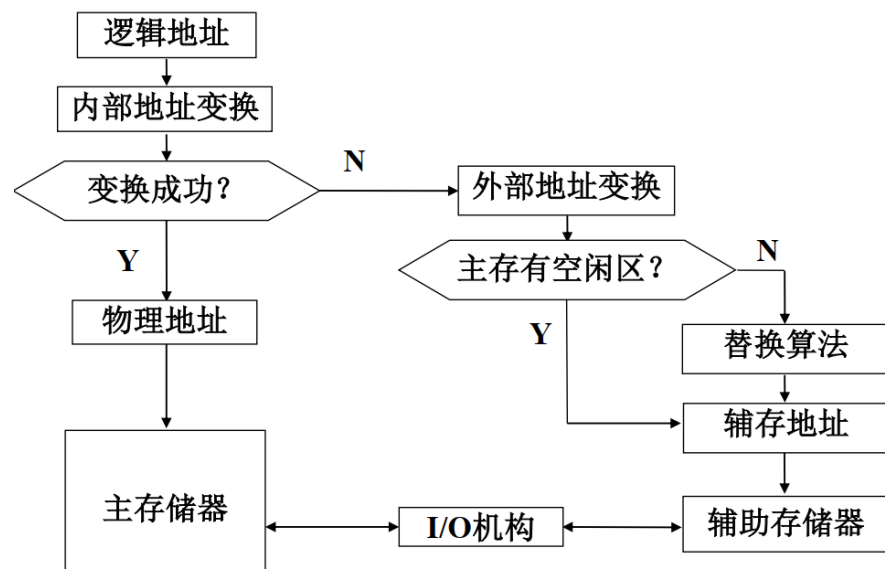


图5.1.1 虚拟存储器工作过程示意图



5.2 分段存储管理

- 5.2.1 分段存储管理的基本思想
- 一个程序由多个模块组成。每个模块都是一个特定功能的独立的程序段。
- **段式管理**：把主存按段分配的存储管理方式。
- 程序模块→段→段描述符→段描述符表→段描述符表寄存器

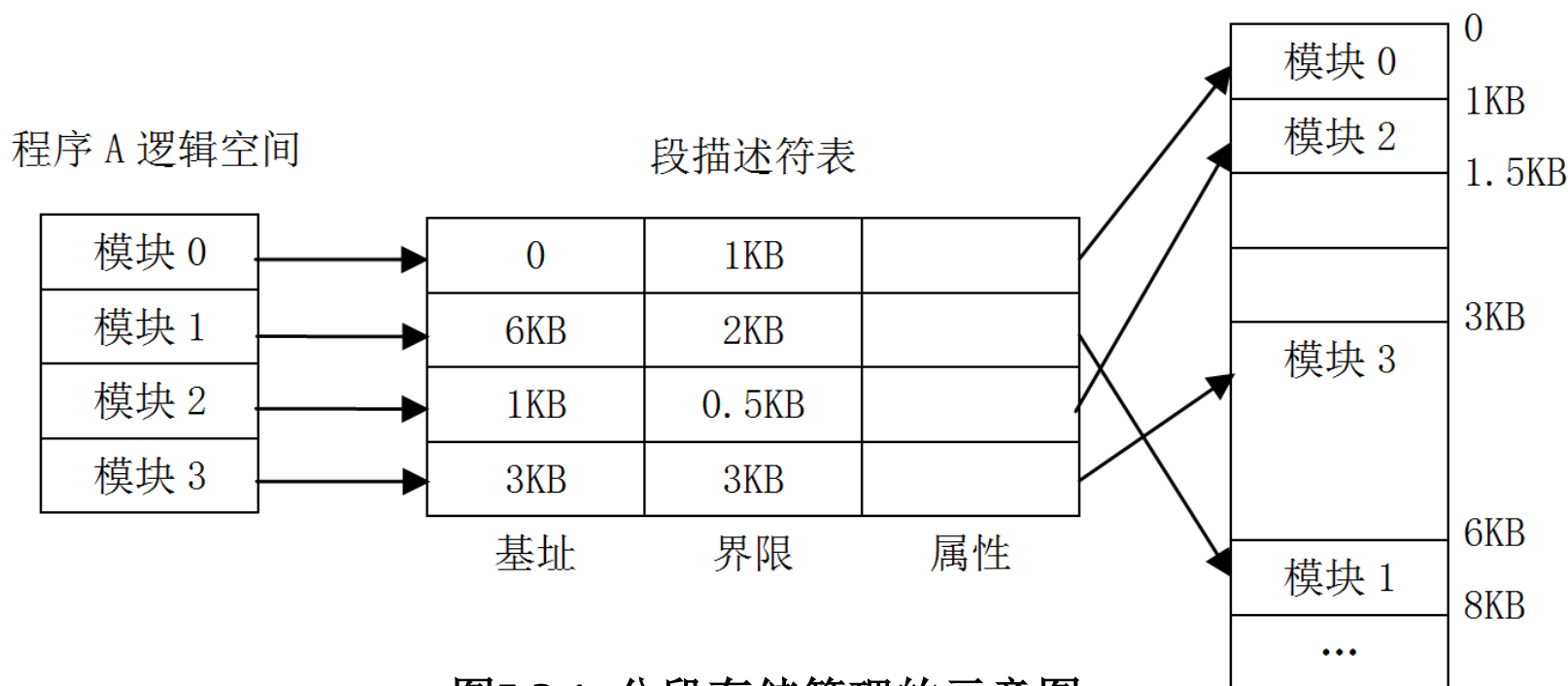


图5.2.1 分段存储管理的示意图



2. 虚拟地址和虚拟地址空间

- Pentium 微处理机在保护模式下的存储器管理单元使用48位的存储器指针，地址空间 2^{48} (64T)B。
- 虚拟地址/逻辑地址构成： 16位段选择符 + 32位偏移量

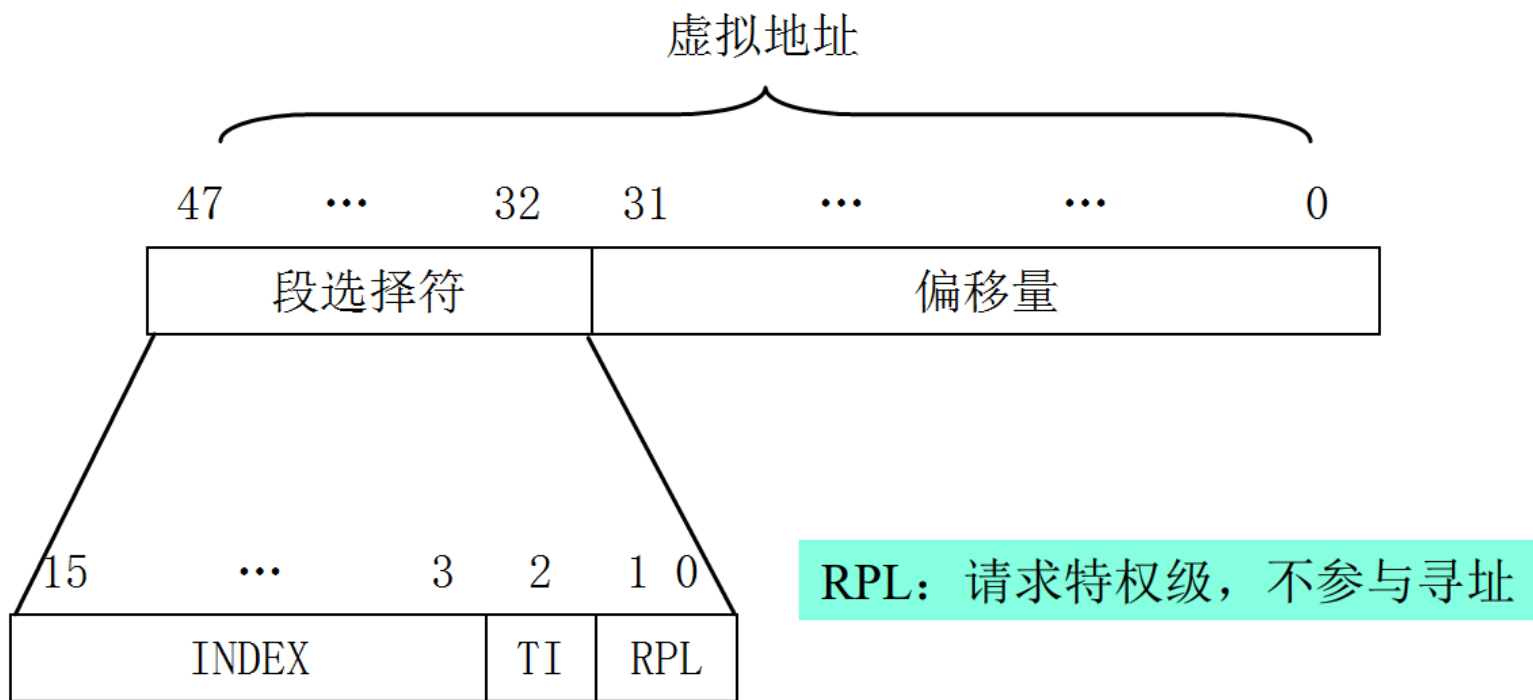


图 5.2.2 保护模式下的存储器指针及段选择符格式



3. 虚实地址转换

- 段选择符 → 段描述符表 → 段描述符 → 段基址 → 偏移量 → 物理地址。

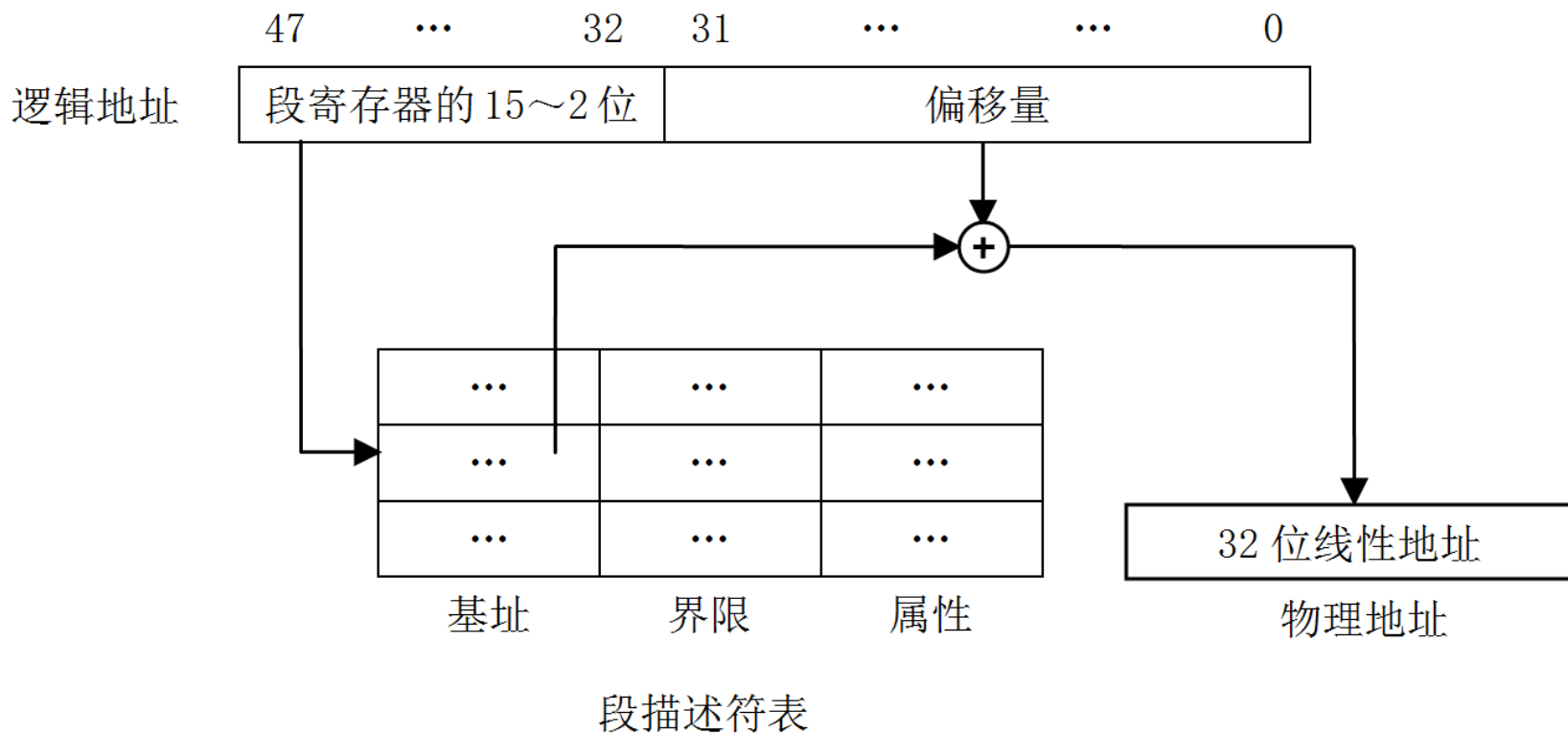


图 5.2.3 虚实地址转换示意图



5.2.2 段描述符

- **段描述符**：用于描述段的基本信息，由8个字节组成。
- 段描述符保存段的属性、段的大小、段在存储器中的位置以及控制和状态信息。
- 段描述符由编译程序、连接程序、装入程序或者是操作系统产生。

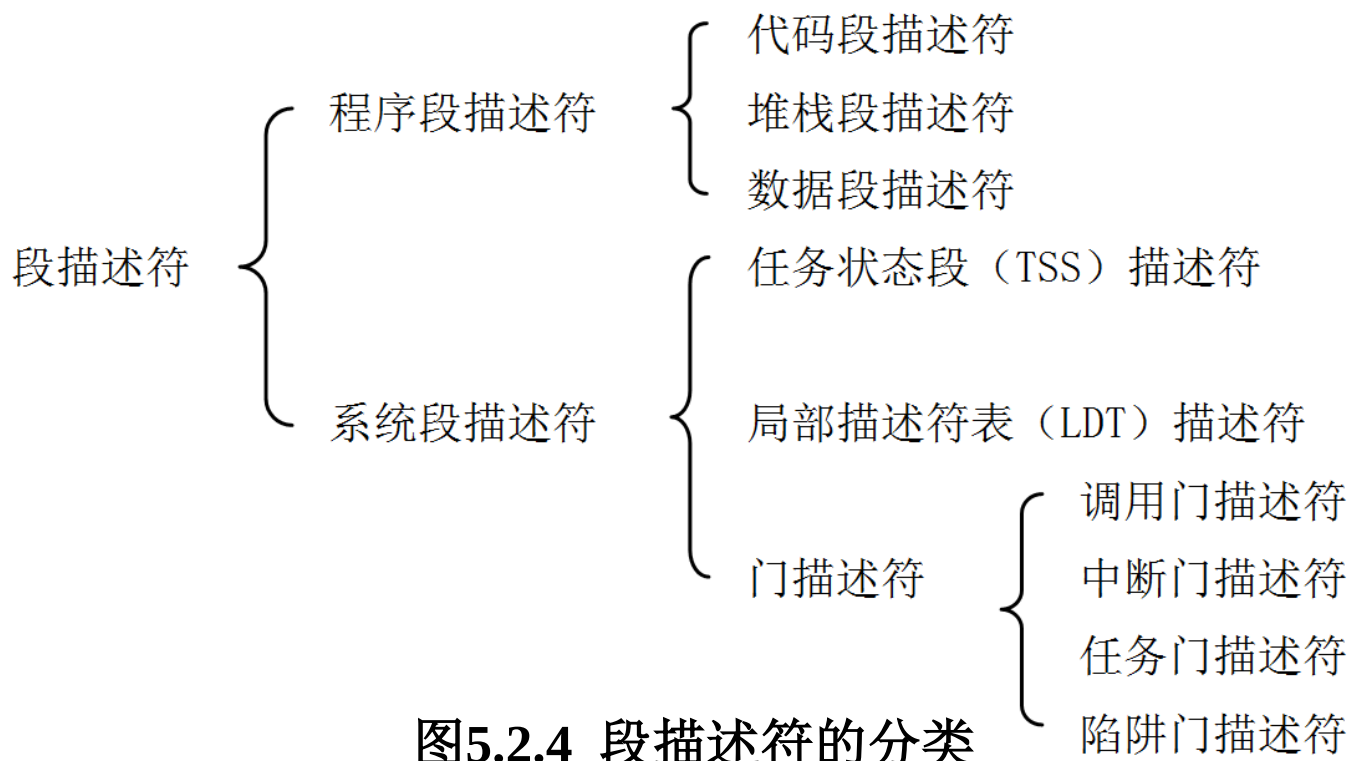


图5.2.4 段描述符的分类



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

基地址： 32位，规定一个段在4GB物理地址空间中的起始位置。



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
			段界限 7~0					0
			段界限 15~8					1
			段基址 7~0					2
			段基址 15~8					3
			段基址 23~16					4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
			段基址 31~24					7

(1) **段界限**：20位，决定了段的长度，该字段的值的单位由“G”位决定。

(2) “G”位称作粒度位，用来确定段界限所使用的长度单位



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

- (1) **粒度位G**：确定段界限所使用的长度单位。
- (2) **G=0**时，段的长度以一个字节为单位。
- (3) **G=1**时，段的长度以**4K**字节为单位。



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

- (1) **分类S**: 区分是系统段描述符还是非系统段描述符。
(2) 当S=0时, 是系统段描述符。
(3) 当S=1时, 是非系统段描述符。



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

- (1) **段存在位P**：该段是否在内存中。
- (2) 当P=1时，表示该段在内存中。
- (3) 当P=0时，表示该段不在内存中。



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

- (1) **系统可用位AVL**：表示系统软件是否可用本段。
- (2) 当AVL=1时，表示系统软件可用本段。
- (3) 当AVL=0时，表示系统软件不能用本段。



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

(1) **特权级DPL**：定义段的特权级。

(2) 2位，有4个特权级：00、01、10、11，称作0级、1级、2级、3级。0级的特权最高，1级次之，3级的特权最低。

(3) 用这个字段定义的特权级去控制对这个段的访问。



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

- (1) **D位/B位**：32/16大小选择。D/B=1，选32位；D/B=0，选16位。
- (2) 在代码段描述符中，指示操作数长度和有效地址长度，D位。
- (3) 在堆栈段描述符中，指示ESP或SP，B位。
- (4) 在数据段描述符中，指示操作数长度。B位



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

兼容位：第6字节的D5位必须是0，以便与将来的处理器兼容。



1. 程序段描述符

表 5.2.1 程序段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
G 粒度	D/B	0	AVL	段界限 19~16				6
段基址 31~24								7

类型TYPE: 在不同的段描述符中有不同的格式。



数据段或堆栈段描述符中的类型TYPE字段

表 5.2.2 数据段或堆栈段描述符中的 TYPE 字段的格式

D7	D6	D5	D4	D3	D2	D1	D0
P	DPL		S=1	E=0	ED	W	A

E可执行位：当E=0时，是数据段或堆栈段。

ED扩展方向位：当ED=0时，向上扩展（地址增加方向），通常用于数据段。
当ED=1时，向下扩展（地址减小方向），通常用于堆栈段。

W可写位：当W=0时，不允许写入。当W=1时，允许写入。

A访问位：当A=0时，该段尚未被访问。当A=1时，该段已被访问



代码段描述符中的类型TYPE字段

表5.2.3 代码段描述符中的类型TYPE字段的格式

D7	D6	D5	D4	D3	D2	D1	D0
P	DPL		S=1	E=1	C	R	A

E可执行位：当E=1时，是代码段。

C一致性位：一致性检查就是采用特权级进行控制。C=0，表示非一致性代码段，此时忽视段描述符的特权值。C=1，表示一致性代码段，需要进行特权级检查。

R可读位：当R=0时，不允许读。当R=1时，允许读。

A访问位：当A=0时，该段尚未被访问；当A=1时，该段已被访问



2. 系统段描述符

表 5.2.4 系统段描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
段界限 7~0								0
段界限 15~8								1
段基址 7~0								2
段基址 15~8								3
段基址 23~16								4
P 存在	DPL 特权级		S 分类		TYPE 类型			5
G 粒度	0	0	0	段界限 19~16				6
段基址 31~24								7

分类S=0，表示为系统段描述符。此时TYPE类型有另的意义。



系统段描述符中的类型TYPE字段的格式

表 5.2.5 系统段描述符中的 TYPE 字段的格式

TYPE	段的类型（用途）	TYPE	段的类型（用途）
0000	未定义（无效）	1000	未定义（无效）
0001	286 TSS 描述符，非忙	1001	TSS 描述符，非忙
0010	LDT 描述符	1010	未定义（保留）
0011	286 TSS 描述符，忙	1011	TSS 描述符，忙
0100	286 调用门描述符	1100	<u>调用门描述符</u>
0101	<u>任务门描述符</u>	1101	未定义（保留）
0110	286 中断门描述符	1110	<u>中断门描述符</u>
0111	286 陷阱门描述符	1111	<u>陷阱门描述符</u>



3. 门描述符

- **门**：一种关卡，用来控制从一段程序到另一段程序或从一个任务到另一个任务的转移。
- **门描述符**：给出一个**逻辑地址**，及转入这个地址的约束，用于控制转入目标代码段的入口点。
- **门描述符包括**：调用门、任务门、中断门和陷阱门。
 - 调用门用于改变特权级别。
 - 任务门用于任务切换。
 - 中断门和陷阱门用于确定中断服务程序。



门描述符的格式

表 5.2.6 门描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
偏移地址 7~0								0
偏移地址 15~8								1
			段选择符 7~0					2
			段选择符 15~8					3
0	0	0	字计数					4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
偏移地址 23~16								6
偏移地址 31~24								7

(1) **段选择符**: 16位, 指出段描述符位置, 索引值。

(2) 任务门送TR, 其他门则送CS。



门描述符的格式

表 5.2.6 门描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
			偏移地址 7~0					0
			偏移地址 15~8					1
			段选择符 7~0					2
			段选择符 15~8					3
0	0	0	字计数					4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
			偏移地址 23~16					6
			偏移地址 31~24					7

偏移地址：32位，指出目标程序的入口偏移量。



门描述符的格式

表 5.2.6 门描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
偏移地址 7~0								0
偏移地址 15~8								1
段选择符 7~0								2
段选择符 15~8								3
0	0	0	字计数					4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
偏移地址 23~16								6
偏移地址 31~24								7

(1) **分类S**: S=0是系统段描述符。S=1是非系统段描述符。

(2) 此处应该S=0，但到底是系统段，还是门，要看类型TYPE。



门描述符的格式

表 5.2.6 门描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
偏移地址 7~0								0
偏移地址 15~8								1
段选择符 7~0								2
段选择符 15~8								3
0	0	0	字计数					4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
偏移地址 23~16								6
偏移地址 31~24								7

- (1) **类型TYPE**：确定门的分类：调用门、任务门、中断门和陷阱门。
- (2) 类型TYPE字段的规则与系统段描述符中的类型TYPE字段的格式完全相同。



门描述符的格式

表 5.2.6 门描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
偏移地址 7~0								0
偏移地址 15~8								1
段选择符 7~0								2
段选择符 15~8								3
0	0	0	字计数					4
P 存在	DPL 特权级		S 分类	TYPE 类型				5
偏移地址 23~16								6
偏移地址 31~24								7

- (1) **P字段**: 表示描述符内容是否有效。
- (2) 当P=0时, 表示描述符内容无效。
- (3) 当P=1时, 表示描述符内容有效。



门描述符的格式

表 5.2.6 门描述符的格式

D7	D6	D5	D4	D3	D2	D1	D0	字节
偏移地址 7~0								0
偏移地址 15~8								1
段选择符 7~0								2
段选择符 15~8								3
0	0	0	字计数				4	
P 存在	DPL 特权级		S 分类	TYPE 类型				5
偏移地址 23~16								6
偏移地址 31~24								7

(1) **字计数**: 指示已有多少字参数要从调用者的堆栈复制到被调用的子程序堆栈 (字计数值)。

(2) 字计数只用于特权级有变化的调用门, 别的门都不用字计数。



5.2.3 全局描述符表及寄存器

- **全局描述符表GDT**: 保存系统使用、各任务共享的段描述符, 只有一个。
- **全局描述符表寄存器GDTR**: 指定了GDT的起始地址
 $48\text{位} = 32\text{位基地址} + 16\text{位界限}$
- **GDT保存的描述符类型**: 除中断门、陷阱门外的各类描述符。
- **GDT寻址可归纳为**:
 1. 段选择符 * 8 + GDTR基址 = 段描述符地址
 2. 段描述符内容 (包括基址) → 相应段cache
 3. cache中段基址 + 虚地址偏移量 = 物理地址
- **访问全局描述符表寄存器**: 指令LGDT、SGDT。



由GDTR确定GDT存储位置和界限

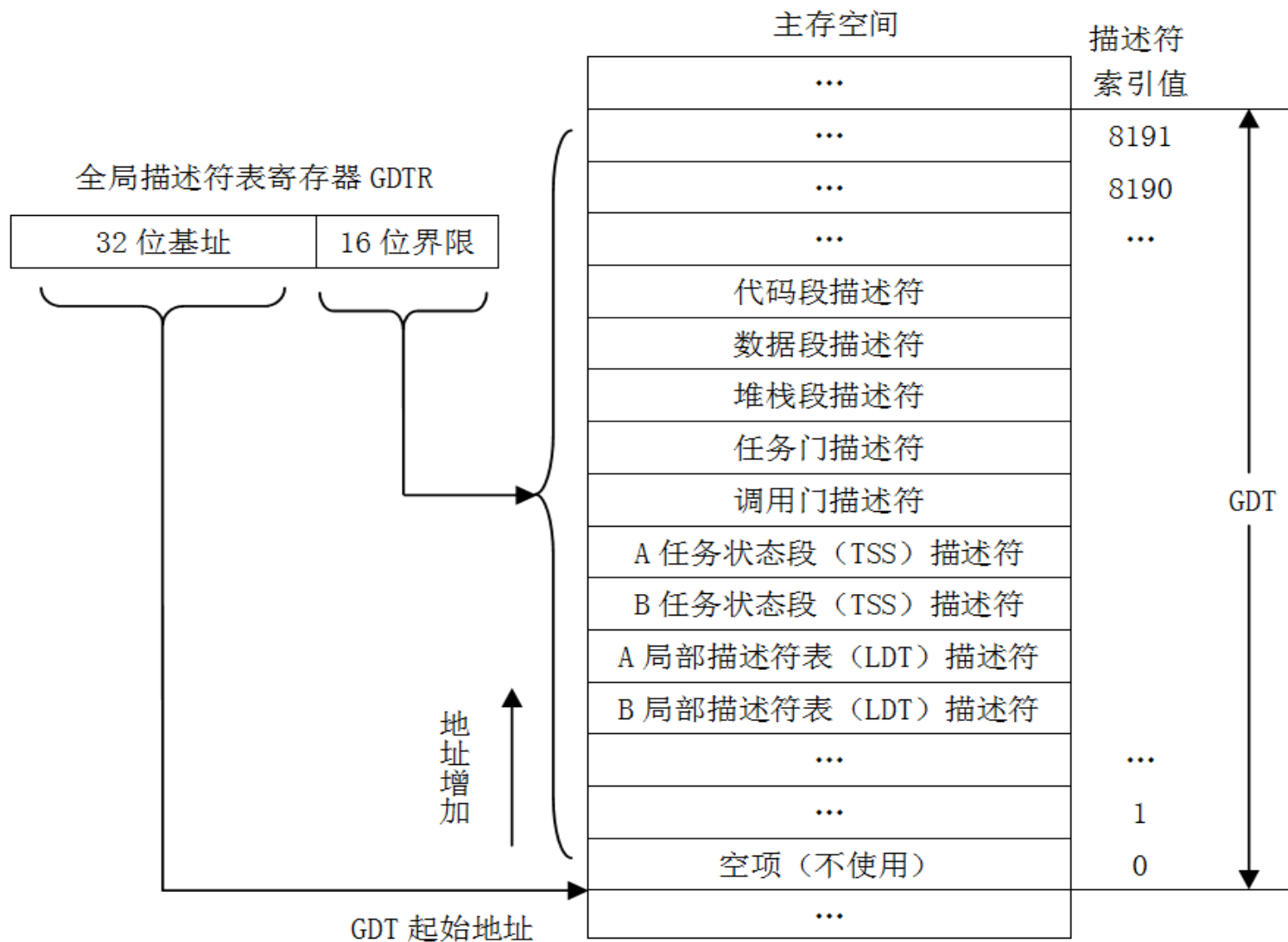


图5.2.5 由GDTR确定GDT存储位置和界限



5.2.4 局部描述符表及寄存器

- **局部描述符表LDT**: 保存某任务使用的段描述符，每个任务一个。
- **LDTR**: 指出LDT的基址
 $16\text{位选择符} + (32\text{位基址} + 20\text{位界限} + 12\text{位属性})$
- LDTR的段选择符确定LDT描述符在GDT中的位置。
- 如果LDTR中装入了段选择符，处理器自动地将相应的LDT描述符从全局描述符表GDT中读出来，并装入LDTR中的cache部分，其中包括LDT基址，从而为当前任务创建一个LDT。



由LDTR确定LDT存储位置和界限

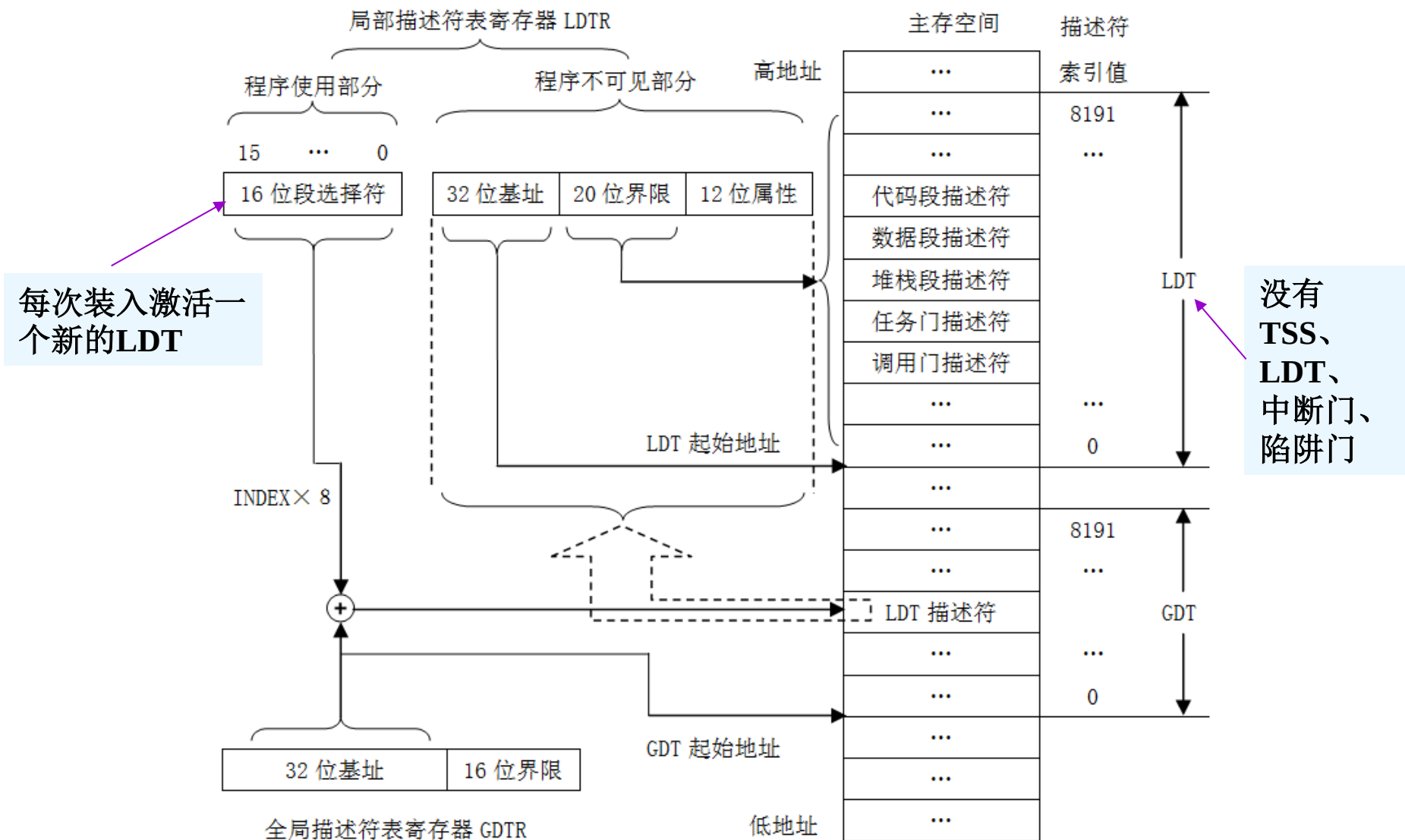


图 5.2.6 由 LDTR 确定 LDT 存储位置和界限



5.2.5 中断描述符表及寄存器

- **中断描述符表IDT:** 保存门描述符，整个系统一个，包括中断门、陷阱门、任务门（通常没有调用门）。
- 门提供了一种将程序控制转移到中断服务程序入口的手段。每个门8个字节，包含服务程序的属性和起始地址。
- **中断描述符表寄存器IDTR:**

48位 = 32位基地址 + 16位界限

IDT按字节计算大小，IDT最大可达64KB（但是Pentium微处理机只能够支持256个中断和异常，最多占用2KB）。
- 中断描述符表IDT中存放的描述符类型均是门描述符。



由IDTR确定IDT存储位置和界限

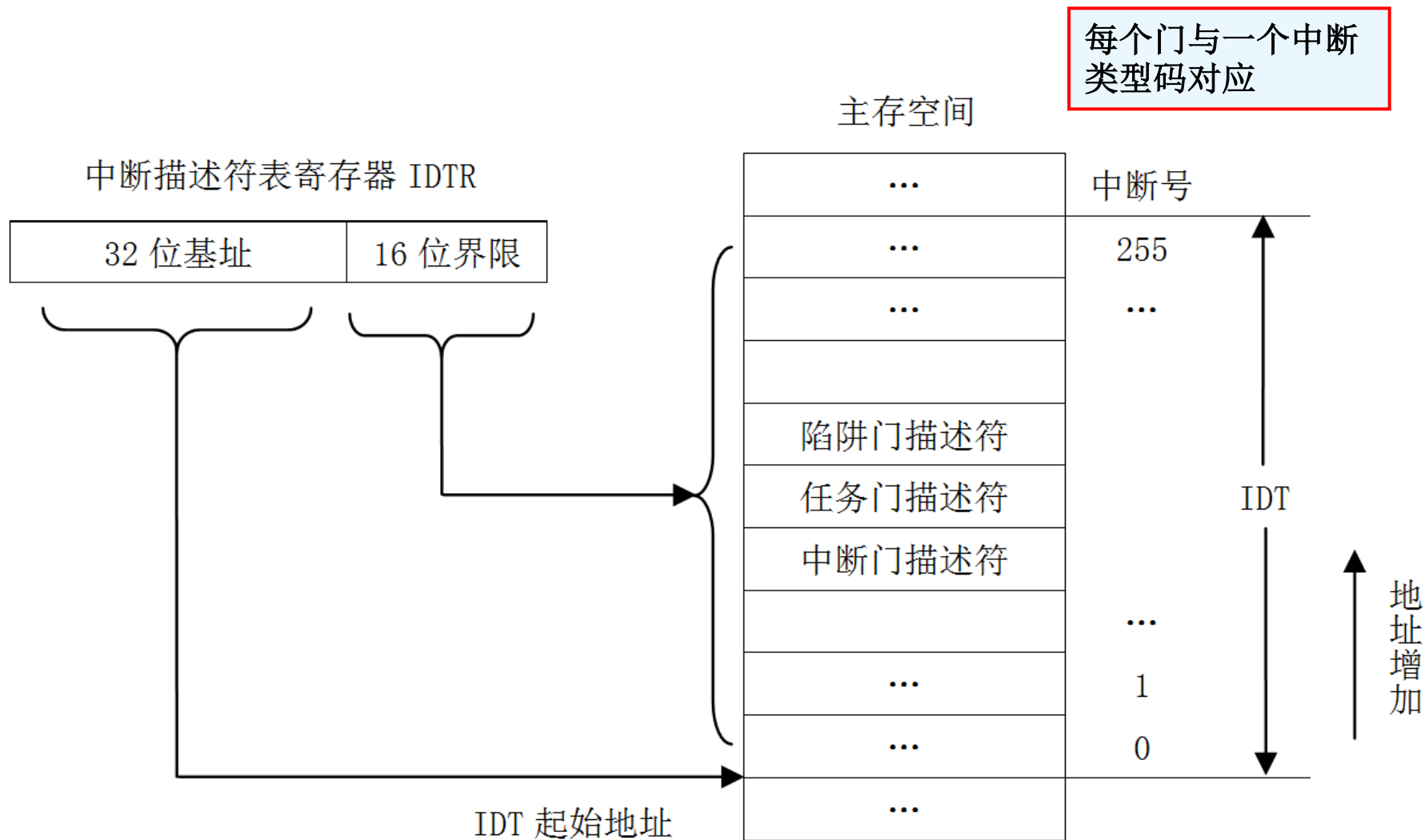


图 5.2.7 由 IDTR 确定 IDT 存储位置和界限



5.2.6 任务状态段及寄存器

- **任务状态段TSS:** 保存现有任务的机器状态（指处理器的工作环境，比如各个寄存器的状态）及其任务间的关联信息。
- 没有数据和代码，每个任务一个。



图 5.2.8 TSS 的格式



TSS的格式

(1) **返回链BACK_LINK**: 是一个段选择符, 把前一个任务的TSS描述符的段选择符(即原来TR中的16位可见部分)转入新任务TSS中, 供任务返回时使用。

(2) 由返回指令IRET将其装入TR寄存器, 从而回到前一个TSS。

	31	...	16	15	...	0	
00	0000 0000 0000 0000				返回链 BACK_LINK		返回链接
04	ESP0						CPL0, 1, 2 的堆栈
08	SS0						
0C	ESP1						
10	SS1						
14	ESP2						
18	SS2						
1C	CR3						地址映射保护
20	EIP						当前任务状态 (寄存器保护)
24	EFLAGS						
28	EAX						
2C	ECX						
30	EDX						
34	EBX						
38	ESP						
3C	EBP						
40	ESI						
44	EDI						
48	0000 0000 0000 0000				ES		地址映射保护
4C	0000 0000 0000 0000				CS		
50	0000 0000 0000 0000				SS		
54	0000 0000 0000 0000				DS		
58	0000 0000 0000 0000				FS		
5C	0000 0000 0000 0000				GS		
60	0000 0000 0000 0000				LDT 选择符		
64	I/O 位图偏移地址				0000000000000000		T
	系统状态缓冲区（用于操作系统）						
	I/O 许可位图						

图 5.2.8 TSS 的格式



TSS的格式

CR3中保存前一个状态的
页目录寄存器的基地址。

	31	...	16	15	...	0	
00	0000 0000 0000 0000				返回链 BACK_LINK		返回链接
04	ESP0						CPL0, 1, 2 的堆栈
08					SS0		
0C	ESP1						
10					SS1		
14	ESP2						
18					SS2		
1C	CR3						地址映射保护
20	EIP						当前任务状态 (寄存器保护)
24	EFLAGS						
28	EAX						
2C	ECX						
30	EDX						
34	EBX						
38	ESP						
3C	EBP						
40	ESI						
44	EDI						
48	0000 0000 0000 0000				ES		地址映射保护
4C	0000 0000 0000 0000				CS		
50	0000 0000 0000 0000				SS		
54	0000 0000 0000 0000				DS		
58	0000 0000 0000 0000				FS		
5C	0000 0000 0000 0000				GS		
60	0000 0000 0000 0000				LDT 选择符		
64	I/O 位图偏移地址				0000000000000000		T
	系统状态缓冲区（用于操作系统）						
	I/O 许可位图						

图 5.2.8 TSS 的格式



TSS的格式

(1) 寄存器保存区域，用于保存通用寄存器、段寄存器、指令指针和标志寄存器。

(2) 每当任务切换时，处理器当前所有寄存器的内容都被保存在TSS的这些单元中，然后将新任务TSS所对应的单元内容装入所有的寄存器。

	31	...	16	15	...	0	
00	0000 0000 0000 0000				返回链 BACK_LINK		返回链接
04	ESP0						CPL0, 1, 2 的堆栈
08					SS0		
0C	ESP1						
10					SS1		
14	ESP2						
18					SS2		
1C	CR3						地址映射保护
20	EIP						当前任务状态 (寄存器保护)
24	EFLAGS						
28	EAX						
2C	ECX						
30	EDX						
34	EBX						
38	ESP						
3C	EBP						
40	ESI						
44	EDI						
48	0000 0000 0000 0000				ES		地址映射保护
4C	0000 0000 0000 0000				CS		
50	0000 0000 0000 0000				SS		
54	0000 0000 0000 0000				DS		
58	0000 0000 0000 0000				FS		
5C	0000 0000 0000 0000				GS		
60	0000 0000 0000 0000				LDT 选择符		地址映射保护
64	I/O 位图偏移地址				0000000000000000		T
	系统状态缓冲区（用于操作系统）						
	I/O 许可位图						

图 5.2.8 TSS 的格式



TSS的格式

(1) 每一个任务都有自己的LDT，这里的“LDT描述符的选择符”就是指该TSS所对应的任务的LDT描述符的选择符。

(2) 在任务切换时，要选择新任务的LDT，需要对原来的LDTR内容进行修改，此时，该域作为LDTR选择符的修改值使用。

	31	...	16	15	...	0	
00	0000 0000 0000 0000				返回链 BACK_LINK		返回链接
04	ESP0						CPL0, 1, 2 的堆栈
08					SS0		
0C	ESP1						
10					SS1		
14	ESP2						
18					SS2		
1C	CR3						地址映射保护
20	EIP						当前任务状态 (寄存器保护)
24	EFLAGS						
28	EAX						
2C	ECX						
30	EDX						
34	EBX						
38	ESP						
3C	EBP						
40	ESI						
44	EDI						
48	0000 0000 0000 0000				ES		地址映射保护
4C	0000 0000 0000 0000				CS		
50	0000 0000 0000 0000				SS		
54	0000 0000 0000 0000				DS		
58	0000 0000 0000 0000				FS		
5C	0000 0000 0000 0000				GS		
60	0000 0000 0000 0000				LDT 选择符		
64	I/O 位图偏移地址			0000000000000000			T
系统状态缓冲区（用于操作系统）							
I/O 许可位图							

图 5.2.8 TSS 的格式



由TR确定TSS存储位置和界限

(1) TR的内容由装任务寄存器指令LTR和存任务寄存器指令STR进行装入和保存。

(2) 也可由保护模式下运行远转移JMP或远调用CALL指令来改变。

(3) 也可来自任务门。

TSS描述符由任务寄存器TR寻址。

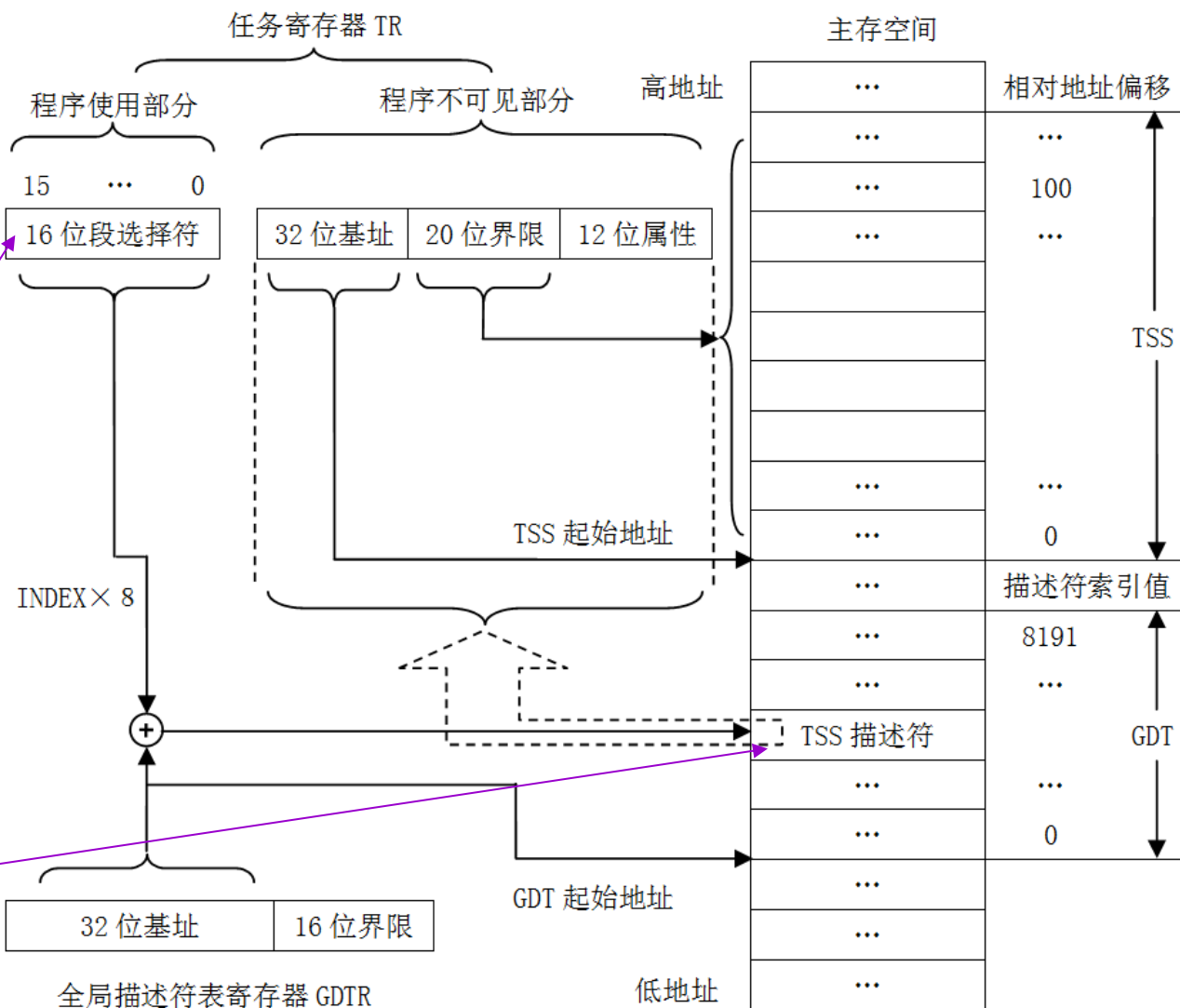


图5.2.9 由TR确定TSS存储位置和界限



5.2.7 段选择符及寄存器

- 在实模式下，段寄存器的16位值是基地址。
- 在保护模式下，每一个段寄存器由两部分组成：
可见部分 + 不可见部分（高速缓存）

16位

64位

(1) 可用传送指令MOV装入，装入的是16位的段选择符。

(2) 段选择符用于识别（选择）在全局描述符表GDT或局部描述符表LDT内登记的段描述符。

只能由处理机装入，用户不能干预



1. 段选择符

- 选择符分为3个字段：

INDEX

索引字段

13位

索引值乘以8就是相对于GDT或LDT首址的偏移量，这个偏移量再加上描述符表的基地址（来自全局描述符表寄存器GDTR，或者局部描述符表寄存器LDTR）就是段描述符在描述符表中的地址。

+

TI

描述符表选择字段

1位

（1）当TI=0时，选择的是全局描述符表GDT。

（2）当TI=1时，选择的是局部描述符表LDT。

+

RPL

请求特权级字段

2位

（1）有4个特权级，00、01、10、11，称作0级、1级、2级、3级。

（2）0级的特权最高，1级次之，3级的特权最低。



2. 段选择符装入段寄存器的操作

- 装入段寄存器的指令有2类：
 - (1) 直接的装段寄存器指令：例如 `MOV DS, AX` 等。
 - (2) 隐含的装段寄存器指令：例如 `CALL FAR SUB1, JMP FAR AA6` 等。



5.3 保护模式下的访问操作与保护机制

- 5.3.1 保护机制的分类
 - 1. 任务间存储空间的保护
 - 任务间的保护是通过每一个任务所专用的LDT描述符实现的。根据LDT描述符，每个任务都有它特定的虚拟空间，因而避免各任务之间的干扰，起到隔离、保护的作用。
 - 2. 段属性和界限的保护
 - 当段寄存器进行加载时，需要进行段存在性检查以及段限检查。
 - 在段描述符中给出了20位的段界限值，每当产生一个逻辑地址时，都要比较偏移量和段限值。一旦偏移地址大于段限值，CPU就终止执行命令，并发出越限异常。由此限制每个程序段只在自己的程序、数据段内运行，不相互干扰。
 - 最后，还要对该段的读写权限进行检查。
 - 3. 特权级与特权级保护
 - 特权级与特权保护是为了支持多用户多任务操作系统，使系统程序和用户的任务程序之间、各任务程序之间互不干扰而采取的保护措施。



3种形式的特权管理

(1) 当前特权级CPL

- CPL是当前正在执行的代码段所具有的访问特权级。
- 每一项任务都是在其代码段描述符所确定的特权级中运行。当前特权级就是任务执行时所处的特权级。例如，一个正在运行的任务的CPL，就是其描述符中访问权限字节的DPL。当前特权级的值一般就是代码段描述符中的DPL。

(2) 描述符特权级DPL

- DPL是段被访问的特权级，保存在该段的段描述符的特权级DPL位。

(3) 请求特权级RPL

- RPL是新装入段寄存器的段选择符的特权级，存放在段选择符的最低两位
- **特权管理规定：**特权级为P的段中存储的数据，只能由特权级高于或等于P的段中运行的程序使用；特权级为P的代码段/过程，只能由在低于或等于P的特权级下执行的任务调用。



5.3.2 数据段访问

(1) 第1次装入DS，访问内存（GDT或LDT），装入DS的Cache。

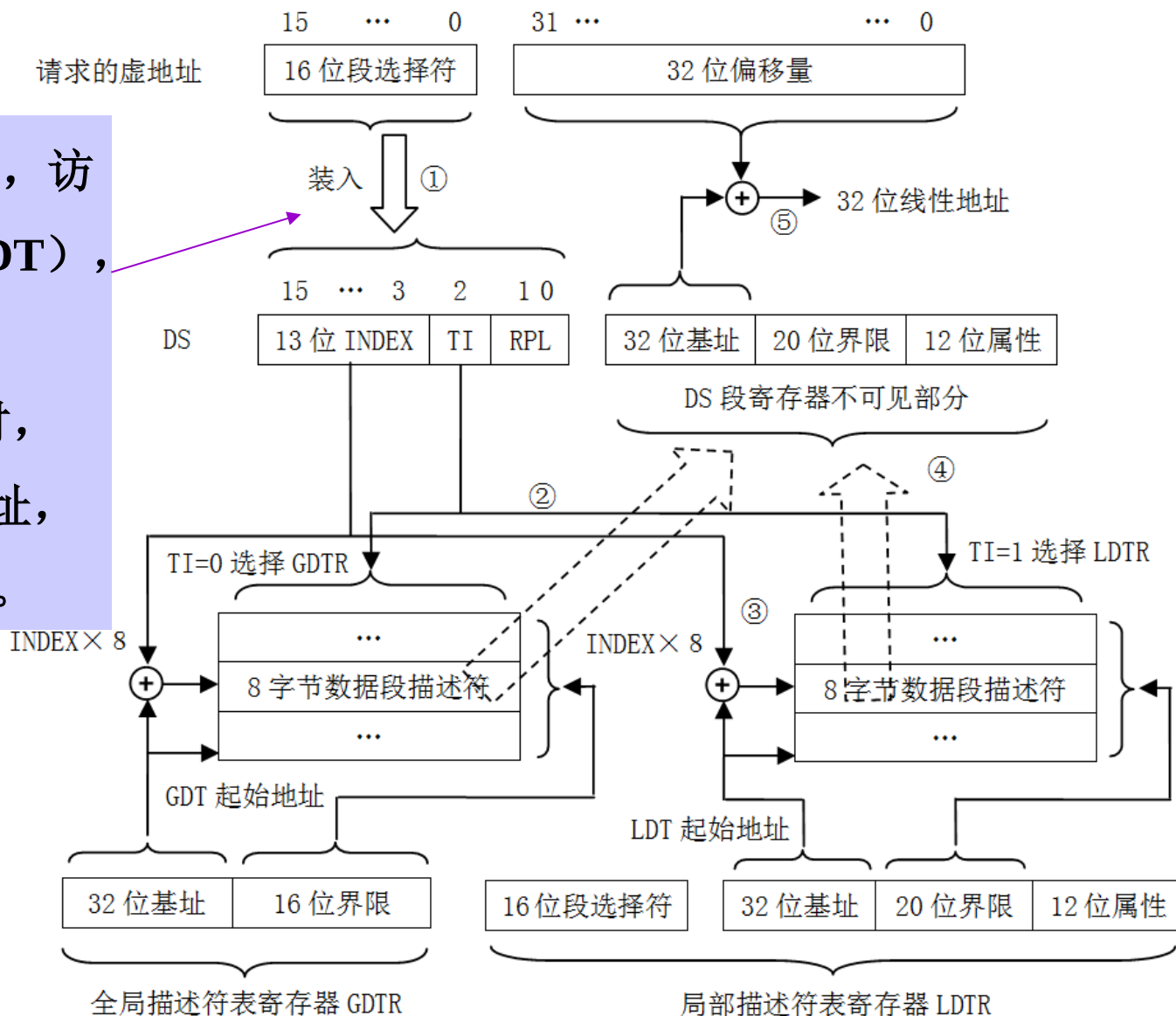
(2) 以后再访问时，从DS的Cache取基址，直接形成线性地址。

要求：

1.能够解释①~⑤步骤。

步骤。

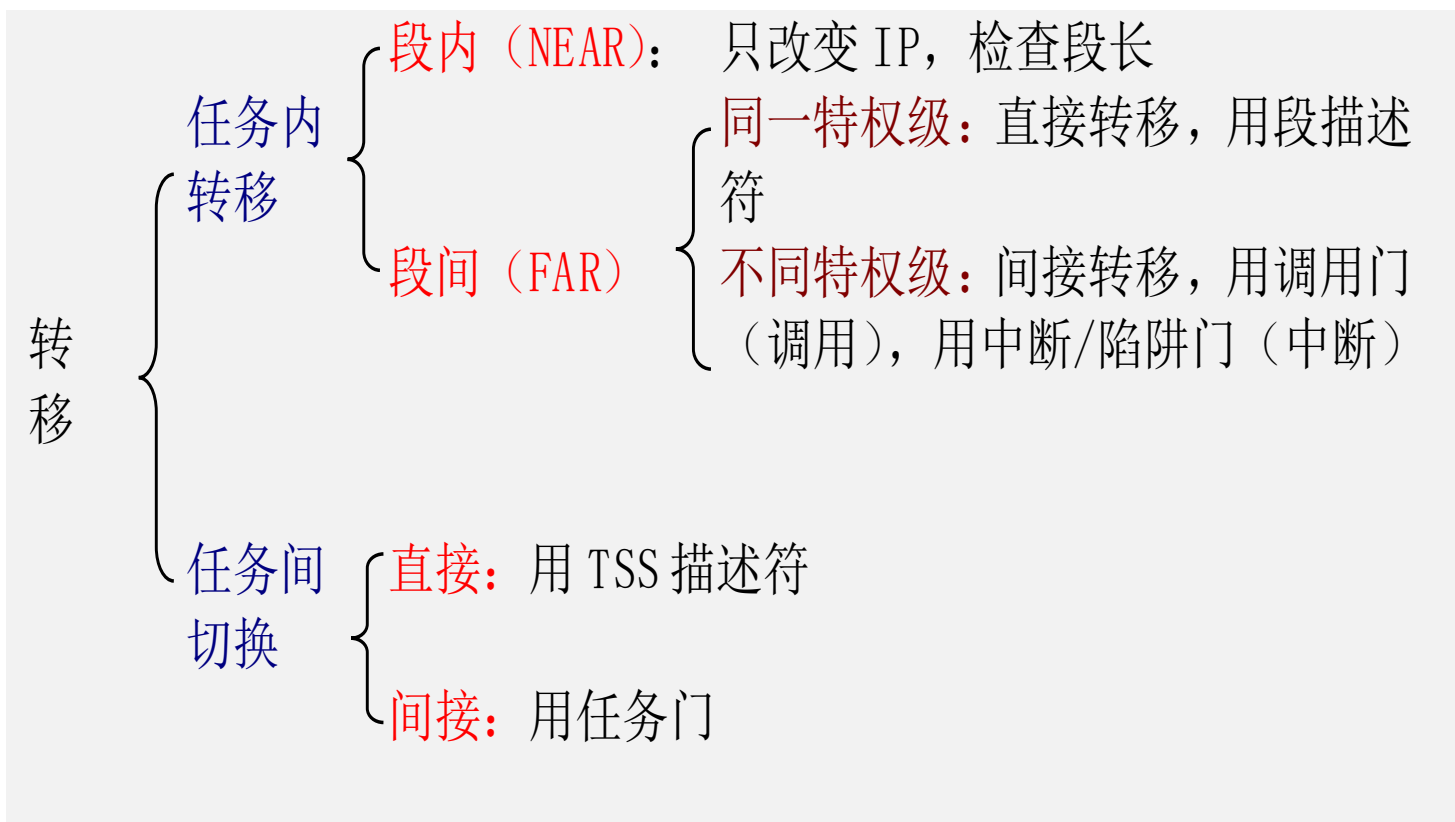
2.能够画出该图





5.3.3 任务内的段间转移

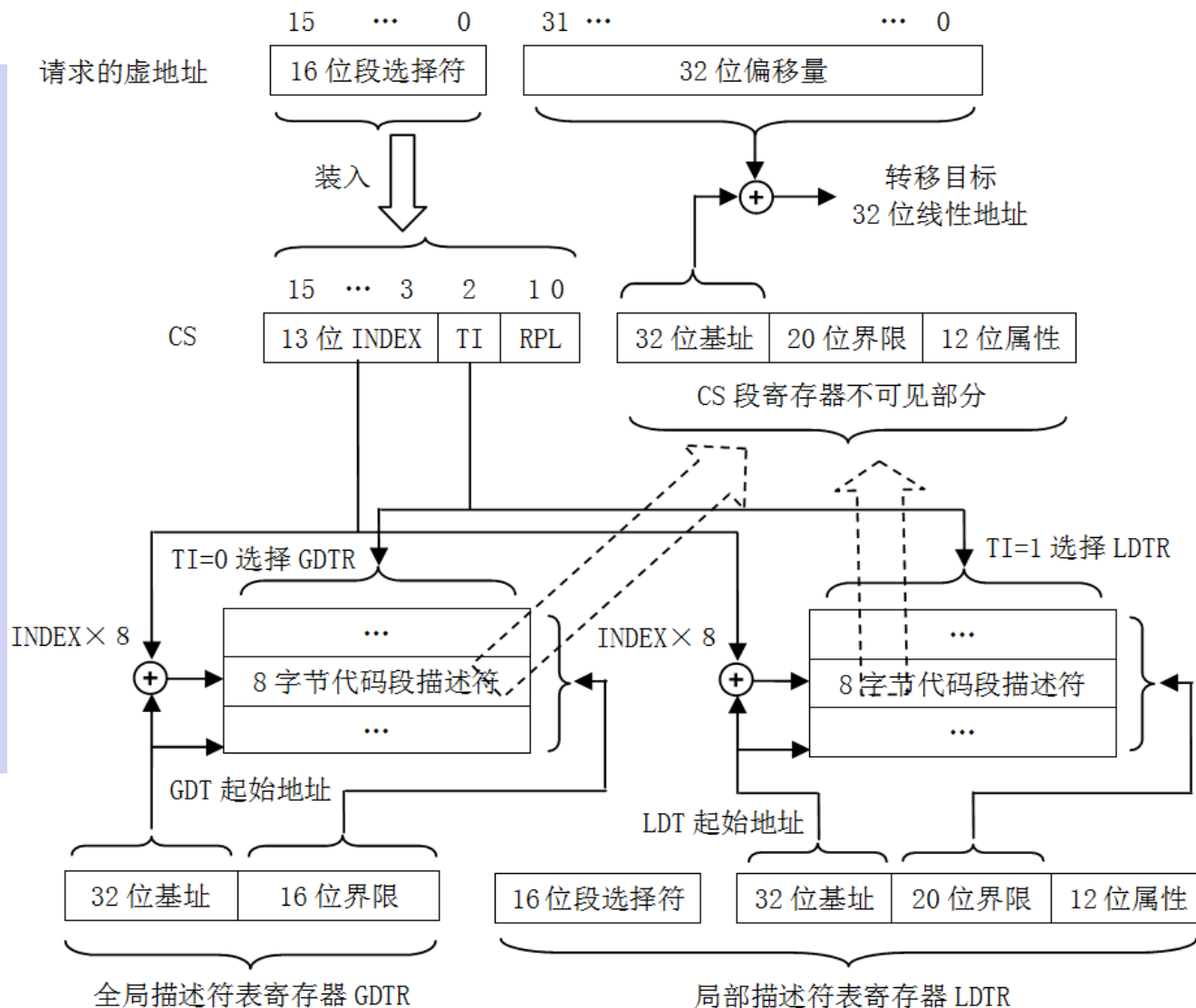
- 当段选择符装入CS寄存器，就会发生段间控制转移。
(段不变时，只检查段限)





1. 段间直接转移的操作过程

- 当送入CS的选择符选择了一个代码段描述符。
- 段间直接转移的操作过程类似访问数据段的操作过程





3. 段间间接转移的操作过程

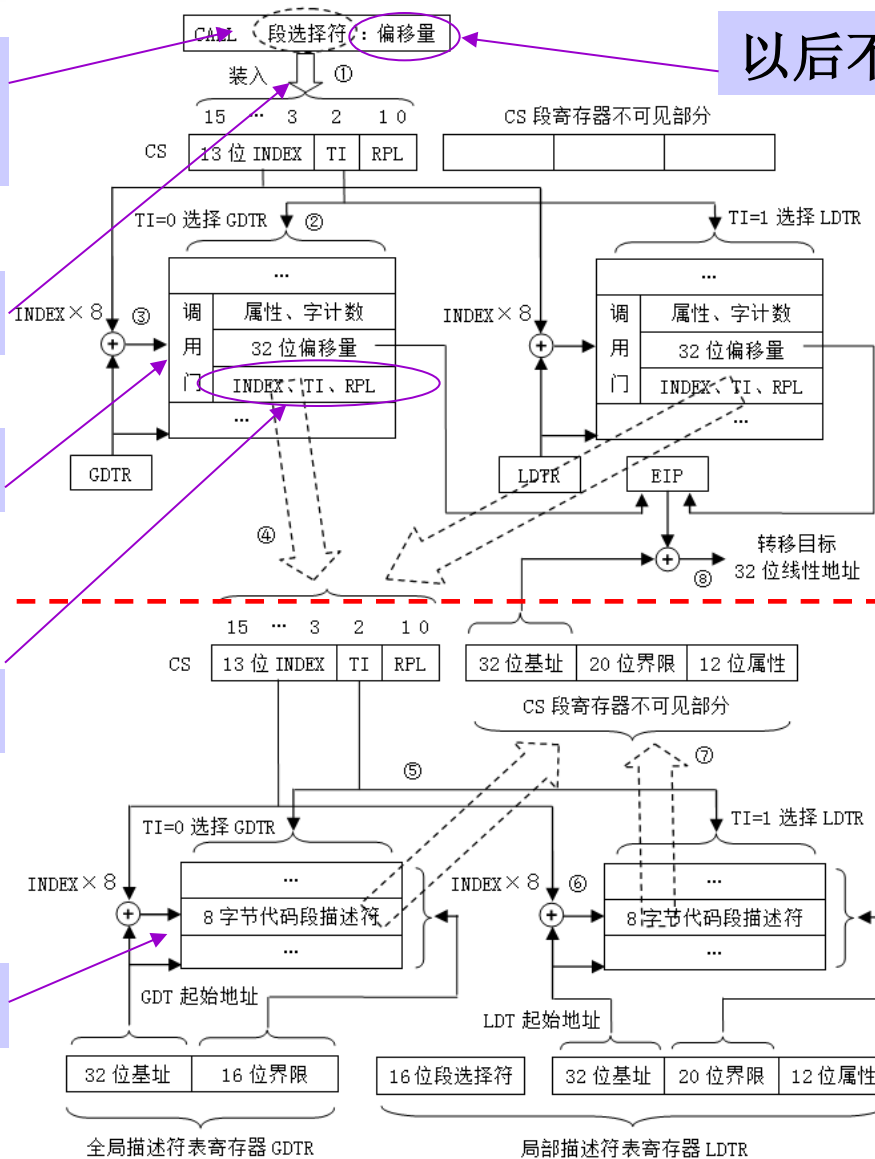
当送入CS的选择符选择了一个调用门描述符。

由指令装入

调用门按数据段保护

目标段的段选择符，装入CS

目标段按代码段保护



以后不用

调用门中的地址为目标地址



5.5 分页存储管理

- 分页是虚拟存储器多任务操作系统另一种存储器管理方法。段的长度是可变的，而页的长度是固定的，比如每页4KB。
- **分页：**将程序分成若干个大小相同的页，各页与程序的逻辑结构没有直接的关系。
- **Pentium**微处理器采用**二级页表方法**对页面进行管理，**第1级页表称作页目录**，页目录中的页目录项指明**第2级页表**中各页表的基址。



5.5.1 页目录与页表

- **页目录基地址寄存器CR3:** 保存页目录的基地址，该基地址起始于任意4KB的边界。
- **页目录:** 由页目录项组成，页目录项包含下一级页表的基址和有关页表的信息。Pentium微处理器中，页目录最多包含1024个页目录项，每个页目录项为4个字节，所以，页目录自身占用一个4KB的页面（存储页）。
- **页表:** 由页表项组成，页表项包含页面（存储页）的基址和有关页面的信息。Pentium微处理器中，页表最多包含1024个页表项，每个页表项为4个字节，所以，页表自身也占用一个4KB的页面（存储页）。



页目录项格式

D31	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
页表基址	AV	AI	L	0	0	0	A	PCD	PWT	U/S	R/W	P	
页表 起始地址	系统保留位 系统可 任意使用						访问	页面 Cache 禁止	页面 写直 达	用户/ 系统	读/写	存在	

20位

图 5.5.1 页目录项的格式

页目录项是页表描述符，
4个字节

0=只读

1=存在



页表项格式

D31	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
页面基址	AV	AI	L	0	0	D	A	PCD	PWT	U/S	R/W	P	
物理页面 起始地址	系统保留位 系统可 任意使用					修改	访问	页面 Cache 禁止	页面 写直达	用户/ 系统	读/写	存在	

20位

如该页多长时间
未被访问

图 5.5.2 页表项的格式

页表项是页描述符，
4个字节



5.5.2 分页转换机制

- 在分页转换机制中，当要访问一个操作单元时，32位线性地址转换为32位物理地址是通过两级查表来实现的。
- 分页机制的工作过程如下：
 - (1) 4KB长的页目录存储在由CR3寄存器所指定的物理地址，此地址常称为根地址。
 - (2) 用线性地址中的最高10位（A31-A22）进行页目录索引。
 - (3) 用线性地址中的A21-A12这10位进行页表索引。
 - (4) 以物理页的起始地址为基址，再加上线性地址的最低12位（A11-A0）页内偏移量，就确定了所寻址的物理单元。



分页转换的工作过程

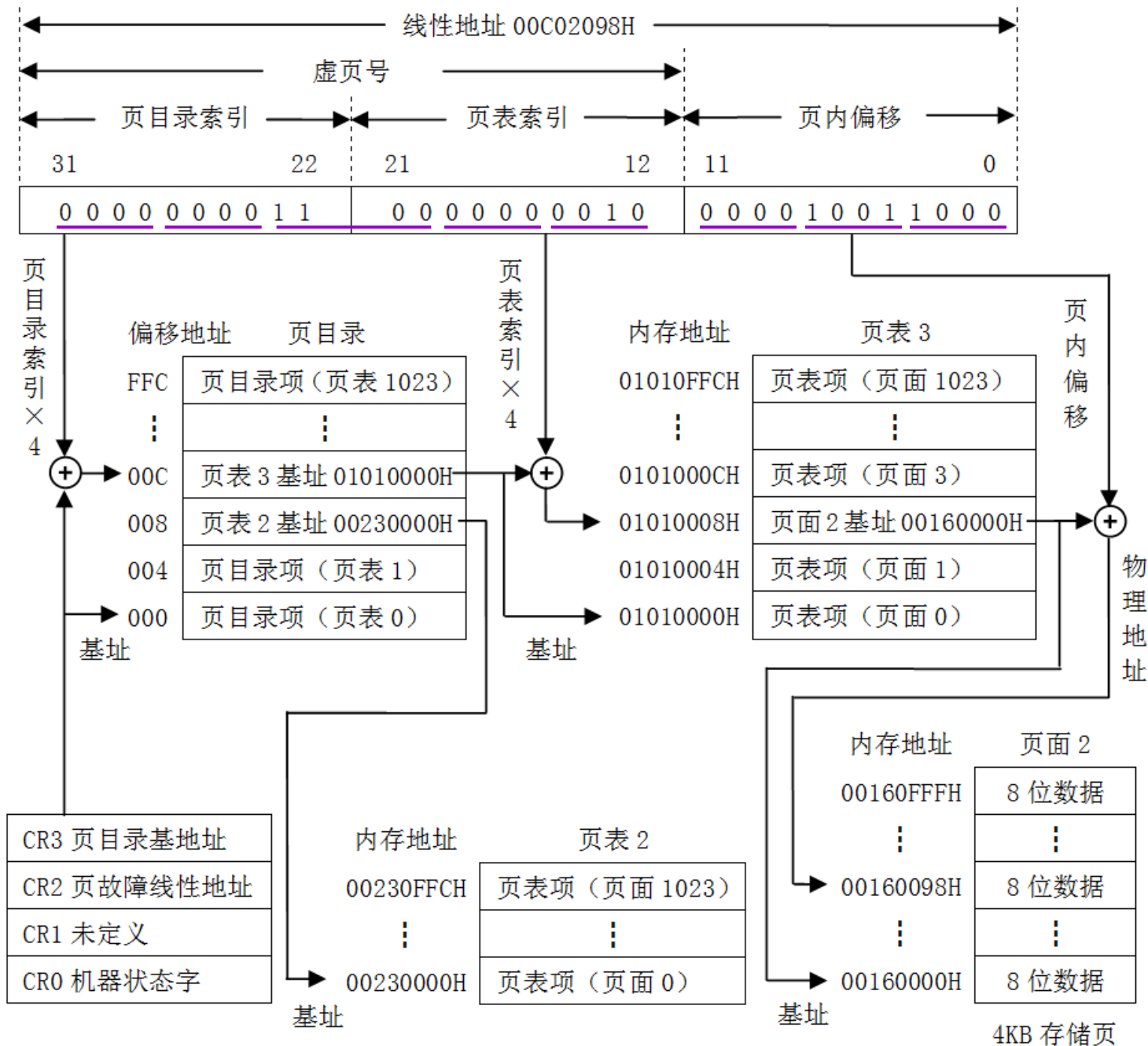


图 5.5.3 分页转换机制示意图



5.6 段页式存储管理的寻址过程

- 在段页式存储管理的寻址过程中，首先将虚地址通过段式存储管理部件转换为线性地址，然后将线性地址通过页式存储管理部件转换为物理地址。
- 在保护模式下，存储器的管理具有分段管理模式、分页管理模式、段页式管理模式等3种，**3种模式的特点：**

(1) 分段不分页。此时，一个任务拥有的最大空间是64T，由分段管理部件将二维虚地址（段选择符，偏移量）转换成一维的32位线性地址，这个线性地址就是物理地址。不分页的好处是：不用访问页目录和页表，地址转换速度快。缺点是：大容量的段调入调出，比较耗时，不够灵活。

(2) 分段分页。由分段管理部件和分页管理部件共同管理。

(3) 不分段分页。此时分段管理部件不工作，分页管理部件工作。程序不提供段选择符，只用32位寄存器地址（作为线性地址）。



段页式存储管理的寻址过程

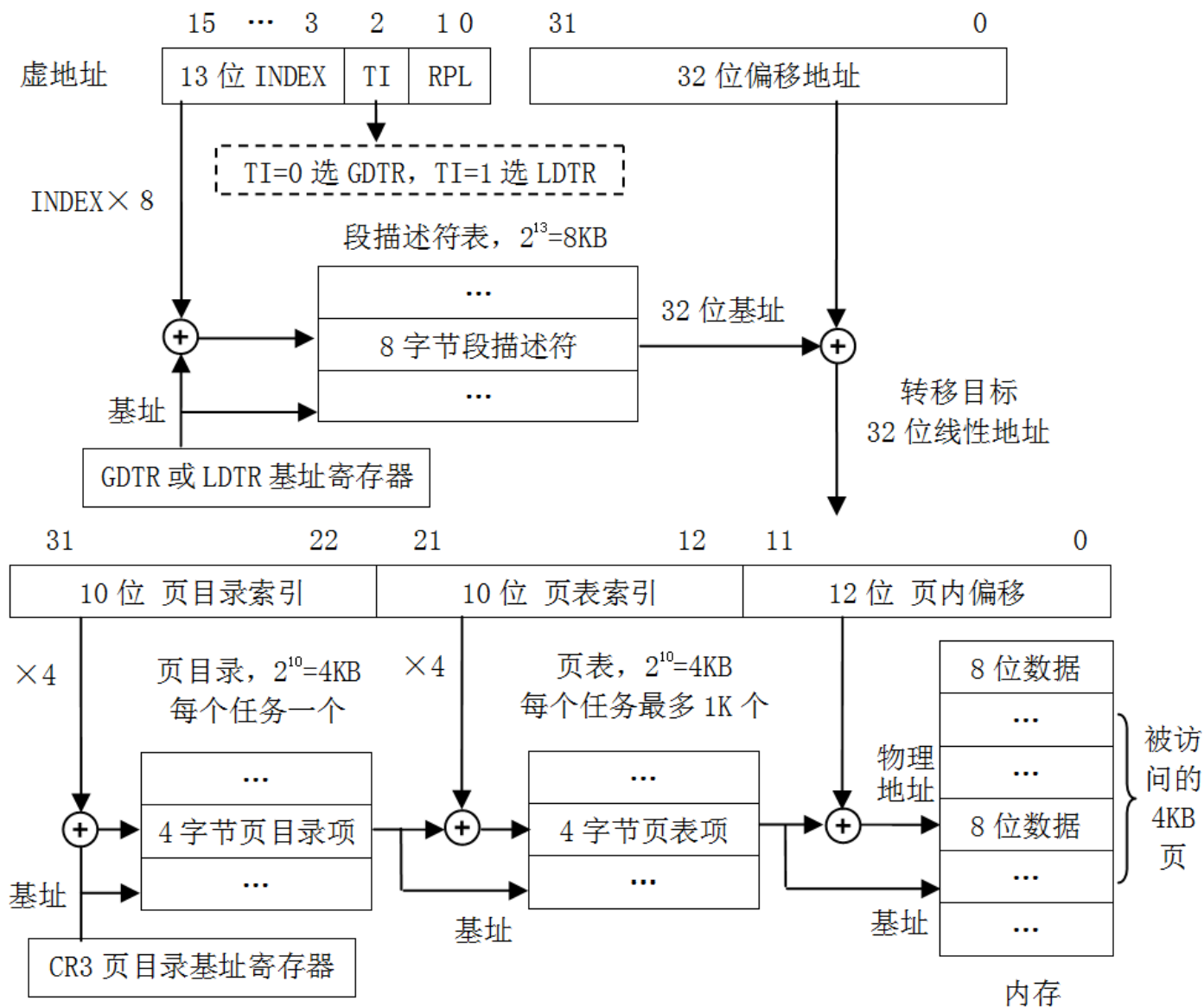


图 5.6.1 段页式存储管理的寻址过程



结 束